

Deep Learning Fundamentals

UW ECE 657A - Core Topic

Mark Crowley

Lecture Outline

- 1 How Simple Neural Networks Work
 - Activation Functions
 - Output Layer
- 2 Neural Networks as Universal Approximators
- 3 Training Neural Networks - Backpropagation
- 4 References and Further Reading

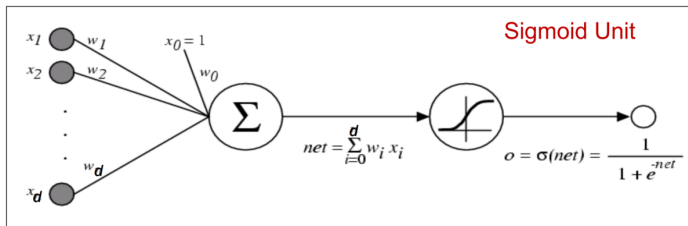
Lecture Outline

- 1 How Simple Neural Networks Work
 - Activation Functions
 - Output Layer
- 2 Neural Networks as Universal Approximators
- 3 Training Neural Networks - Backpropagation
- 4 References and Further Reading

Properties of Neural Networks

- The hidden layer will have a number of units (design parameter).
- The hidden layers allow extracting more complex features and facilitates solving complex problems.
- Connection between the units (neurons) of all the layers can be **forward**, backward or both.
- Units can be fully or partially connected.
- Different arrangements make different types or models of the NN.
- Each unit will have a **activation function** (a thresholding function) and connections between the units that have weights

The Basic Neural Unit



$$o(x) = \sigma(w^T x_i) = \sigma(w_0 + \sum_i w_i x_i) = \frac{1}{1 + \exp(-(w_0 + \sum_i w_i x_i))}$$

$$\text{Differentiable: } \frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

Question: Why σ ? Is this the best function to use?

Net Activation

$$net_j = \sum_{i=1}^d x_i w_{ji} + w_{j0} = \sum_{i=0}^d x_i w_{ji}$$

where:

- i indexes the inputs units
- j indexes the hidden units
- W_{ji} denotes the weights on input to hidden layer (“synaptic weights”)

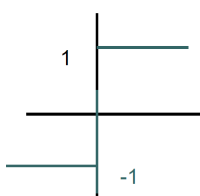
Hidden Layer

- Each hidden unit emits an output that is a nonlinear **activation function** of its net activation.

$$y_j = f(\text{net}_j)$$

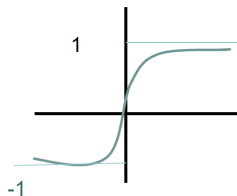
- This is essential to a neural network's power, if it were linear then it would all become nothing more than *linear regression*.
- The output is thresholded through this nonlinear activation function.

Activation Functions



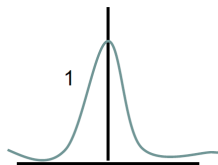
a) Threshold

$$F_T(x) = \begin{cases} 1 & \text{if } x > \tau \\ -1 & \text{otherwise} \end{cases}$$



b) Sigmoid

$$F_S(x) = \frac{1}{(1 + e^{-cx})}$$

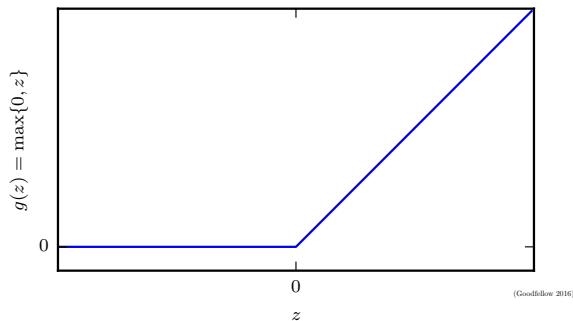


c) Gaussian

$$F_G(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}$$

- tanh was another common function.
- sigmoid is now discouraged except for final layer to obtain probabilities. Can **over-saturate** easily.
- ReLU is now usually the standard activation function to use.

Rectified Linear Activation



- **Rectified Linear Units (ReLU)** - this has become standard $\max(0, net_j)$
 - strong signals are always easy to distinguish
 - most values are zero, derivative is mostly zero
 - they do not saturate as easily as sigmoid

But Wait!...There's more!

- **Exponential linear units** - evidence that they perform better than ReLU in some situations.
- **Leaky ReLU** - remove the extremeness of zero in ReLU. Why do we need to do that?
- **Scaled Exponential Linear Unit (SeLU)** - a threshold λ and a nonlinear scale α create a “leaky” LU which is softer than LReLU.
- Read more <https://mlfromscratch.com/activation-functions-explained/>

Output Layer

Regardless of activation function, the output layer then combines all the previous hidden layers.

$$net_k = \sum_{j=1}^{n_H} y_j w_{kj} + w_{k0}$$

where:

- k indexes the output units
- n_H denotes the number of hidden units in the previous layer

Output Layer

The output layer adds *another* level of thresholding, usually nonlinear as well.

$$z_k = f(\text{net}_k)$$

$f(\text{net}_k)$ could be any thresholding function such as the previous activation functions.

Lecture Outline

- 1 How Simple Neural Networks Work
 - Activation Functions
 - Output Layer
- 2 Neural Networks as Universal Approximators
- 3 Training Neural Networks - Backpropagation
- 4 References and Further Reading

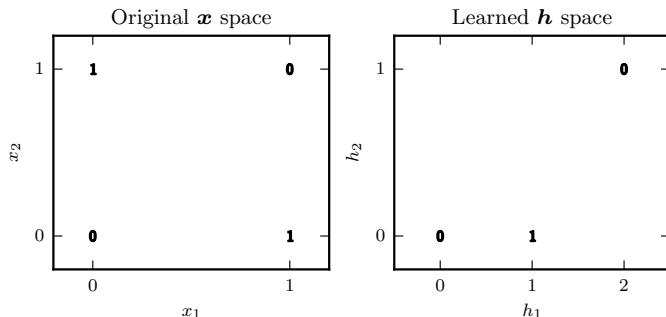
Outline

- 1 How Simple Neural Networks Work
- 2 Neural Networks as Universal Approximators
- 3 Training Neural Networks - Backpropagation
- 4 References and Further Reading

Failing on the XOR Problem

How is XOR a hard problem? **It is if you're too linear.** XOR is the simplest **nonlinear** classification problem.

- Two binary features x_1 and x_2 and four patterns to be assigned the correct labels True or False



Neural Nets Can Model XOR Problem

Use a simple `sign` activation function for hidden and output units:

$$\begin{aligned} f(\text{net}) &= 1 \text{ if } \text{net} \geq 0 \\ &= -1 \text{ if } \text{net} < 0 \end{aligned}$$

- First hidden unit:
 - y_1 computes $x_1 + x_2 + 0.5 = 0$
 - $y_1 = 1$ if $\text{net}_1 \geq 0$, $y_1 = -1$ otherwise
- Second hidden unit:
 - y_1 computes $x_1 + x_2 - 1.5 = 0$
 - $y_2 = 1$ if $\text{net}_2 \geq 0$, $y_2 = -1$ otherwise
- single output unit z_1 :
 - emits $z_1 = 1$ iff $y_1 = 1$ and $y_2 = 1$.
 - y_1 models AND gate
 - y_2 models OR gate
 - z_1 models y_1 AND NOT y_2

Full Neural Network Discriminant Function

The class of decision functions that can be described by a three-layer neural network are:

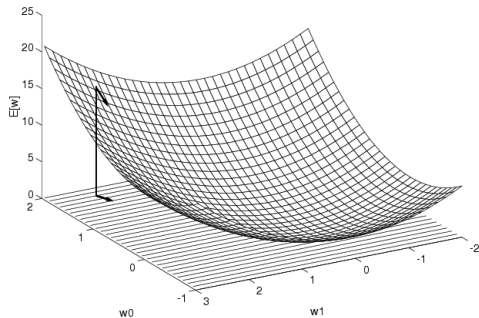
$$g_k(x) = z_k = f \left(\sum_{j=1}^{n_H} w_{kj} f \left(\sum_{i=1}^d w_{ji} x_i + w_{j0} \right) + w_0 \right)$$

- The **Universal Approximation Theorem** [Hornik, 1989] shows that this covers any possible decision function mapping continuous inputs to a finite set of classes to any desired level of accuracy.
- But it does not say how many hidden units would be needed.
- In general, for a single layer it could be exponential (degrees of freedom).
- But, using more layers can reduce the number of hidden units needed and make it more generalizable.

Lecture Outline

- 1 How Simple Neural Networks Work
 - Activation Functions
 - Output Layer
- 2 Neural Networks as Universal Approximators
- 3 Training Neural Networks - Backpropagation
- 4 References and Further Reading

Gradient Descent



(Slides based on Tom Mitchell ML Course, CMU, 2010)

Error Function: Mean Squared Error, cross-entropy loss, etc.

Gradient: $\nabla E[\mathbf{w}] = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_d} \right]$

Training Update Rule: $\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$ where η is the training rate.

Note: For regression, others, this gradient is convex. In NNs it is not. **So we must solve iteratively**

Incremental Gradient Descent

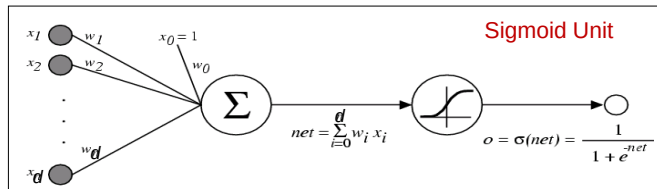
Let error function be : $E_I[w] = \frac{1}{2}(y^I - o^I)^2$

Do until satisfied:

- For each training example I in D
 - ① Compute the gradient $\nabla E[w]$
 - ② update weights : $w = w - \eta \nabla E[w]$

Note: can also use batch gradient descent on many points at once.

Error Gradient for a Sigmoid Unit



$$\begin{aligned}
 \frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{l \in D} (y_l^l - o_l^l)^2 \\
 &= \frac{1}{2} \sum_l \frac{\partial}{\partial w_i} (y_l^l - o_l^l)^2 \\
 &= \frac{1}{2} \sum_l 2(y_l^l - o_l^l) \frac{\partial}{\partial w_i} (y_l^l - o_l^l) \\
 &= \sum_l (y_l^l - o_l^l) \left(-\frac{\partial o_l^l}{\partial w_i} \right) \\
 &= - \sum_l (y_l^l - o_l^l) \frac{\partial o_l^l}{\partial net^l} \frac{\partial net^l}{\partial w_i}
 \end{aligned}$$

But we know:

$$\frac{\partial o_l}{\partial net^l} = \frac{\partial \sigma(net^l)}{\partial net^l} = o_l^l (1 - o_l^l)$$

$$\frac{\partial net^l}{\partial w_i} = \frac{\partial (\vec{w} \cdot \vec{x}^l)}{\partial w_i} = x_i^l$$

So:

$$\frac{\partial E}{\partial w_i} = - \sum_{l \in D} (y_l^l - o_l^l) o_l^l (1 - o_l^l) x_i^l$$

(Slides from Tom Mitchell ML Course, CMU, 2010)

Backpropagation Algorithm

We need an iterative algorithm for getting the gradient efficiently.

For each training example:

- ① **Forward propagation**: Input the training example to the network and compute outputs
- ② **Compute** output units errors:

$$\delta_k^l = o_k^l(1 - o_k^l)(y_k^l - o_k^l)$$

- ③ **Compute** hidden units errors:

$$\delta_h^l = o_h^l(1 - o_h^l) \sum_k w_{h,k} \delta_k^l$$

- ④ **Update** network weights:

$$w_{i,j} = w_{i,j} + \Delta w_{i,j}^l = w_{i,j} + \eta \delta_j^l o_i^l$$

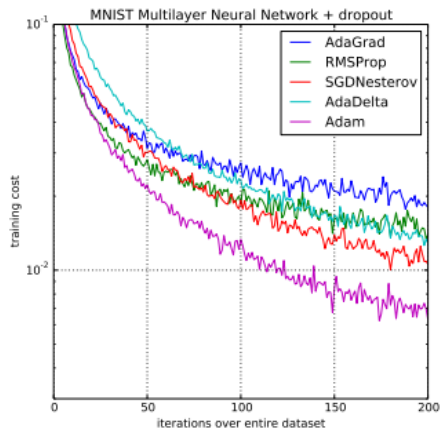
More about Backpropagation

- Still underlies all of Deep Learning
- Can be easily performed on multidimensional outputs.
- No guarantee of global optimal solution, only local, but often good enough.
- Can be parallelized very well, most deep learning tools now do this on GPUs automatically.
- Remember that it is minimizing errors on *training* examples, ability to generalize relies on network structure, training approach, other factors.

Other Optimizers

- **SGD (Stochastic Gradient Descent)**: Can perform best but various additions need to be made and learning rate need to be correctly tuned.
- **Adam (adaptive moment estimate)**: For relatively large datasets Adam is very robust. Converges quickly and gives pretty good performance.
 - keeps track of gradient learning rates and momentum *per dimension*
 - uses variance (2nd moment) rather than means and scales steps based on recent magnitudes
- **L-BFGS** : Converges faster and with better solutions on small datasets. Very memory efficient, scales linearly with variables. Estimates the inverse Hessian matrix (2nd order derivatives)
- See a good overview of the best methods here: [Rud17]
(<https://arxiv.org/abs/1609.04747>)

Adam Optimizer



Lecture Outline

- 1 How Simple Neural Networks Work
 - Activation Functions
 - Output Layer
- 2 Neural Networks as Universal Approximators
- 3 Training Neural Networks - Backpropagation
- 4 References and Further Reading

References



Sebastian Ruder, *An overview of gradient descent optimization algorithms*, [arXiv:1609.04747 \[cs\]](#) (2017), [arXiv: 1609.04747](#).